

Prompt Engineering

Mehmet Ali Akyol, PhD

Ankara Medipol University

October 31, 2025

Week 3: Principles of Good Prompt Design

Where We Are

- Week 1: What is prompt engineering? Use cases, ethics, and course setup.
- Week 2: How LLMs work (non-technical): tokens, embeddings, and transformers.
- **This week:** Practical prompting patterns that make models more reliable and useful in your domain.

Weekly Learning Objectives

- Identify and apply core prompting patterns (zero-shot, few-shot, role, structure, step-by-step).
- Write prompts that specify audience, tone, constraints, and output format.
- Iterate prompts using critique-and-revise loops to improve results.
- Avoid common anti-patterns that cause ambiguity, bias, or hallucination.
- Use guardrails and structured outputs to increase reliability.

Key Concepts

- Task framing and intent
- Zero-shot vs. few-shot prompting
- Role prompting (persona, audience, tone)
- Delimiters and structured output
- Step-by-step and reflection prompting
- Retrieval-augmented prompting (using provided context)
- Iterative refinement (critique → improve)
- Anti-patterns and safety considerations

Anatomy of a Good Prompt

- **Task:** What to do; success criteria in 1–2 bullets.
- **Context:** Facts, definitions, excerpts; what to ignore.
- **Role/Audience:** Persona and target reader; tone/register.
- **Constraints:** Length, format, banned topics, sources.
- **Output Spec:** Headings, JSON schema, table columns.
- **Evaluation:** Checks, rubric, citations, confidence.

Prompt Checklist (Use Before You Run)

- Is the goal explicit and testable?
- Is audience and tone specified?
- Are inputs delimited and labeled clearly?
- Does the output spec match downstream needs?
- Are safety boundaries and fallbacks included?
- Did you request brief reasoning or self-checks where useful?

Prompting Principles

- **Be explicit:** State the task, constraints, and success criteria.
- **Specify the audience:** Student, patient, executive, developer, etc.
- **Constrain the output:** Length, bullets vs. prose, JSON schema, table.
- **Provide context:** Definitions, data, or excerpts to ground the answer.
- **Ask for reasoning when helpful:** “Show steps” or “explain briefly why”.
- **Iterate:** Review the output, then refine the prompt.

Zero-shot vs. Few-shot

- **Zero-shot:** Just the task, no examples. Good for simple, well-known tasks.
- **Few-shot:** Provide 1–3 high-quality examples to steer style or format.
- Tip: Keep examples short and representative; highlight edge cases sparingly.

Role Prompting (Persona, Audience, Tone)

- Set a role: *“You are a patient educator”* or *“You are a senior data engineer”*.
- Specify audience and tone: *“for first-year students, friendly and concise”*.
- Clarify scope: what to include/avoid; reading level; domain jargon threshold.

Delimiters and Structure

- Use clear delimiters to separate instructions and data, e.g., triple backticks.
- Request structured output for downstream use (JSON, CSV, bullet list).

Example

Task: Extract key facts as JSON.

Text:

```
'''
```

The Berlin Wall fell in 1989. It had divided East and West Berlin.

```
'''
```

Output JSON schema: {"year": number, "event": string, "location": string}

Only output valid JSON.

Structured Output: Validation Tricks

- Ask model to *restate the schema* before output.
- Include allowed values/enums, number ranges, and required fields.
- Provide an "example" object and a short "counterexample".
- Fail gracefully: "If you cannot comply, return `VALIDATION_ERROR`".

extbfMini-Schema

```
{"title": string, "level": oneOf["A1","B1","C1"], "bullets": s
```

Step-by-step and Reflection

- Ask for steps or a brief plan before the final answer (helps correctness and clarity).
- Reflection prompts: *Before finalizing, list 2 potential mistakes and fix them*
- Keep reasoning concise to reduce verbosity; prefer targeted checks over generic `think step-by-step`.

Deeper Reasoning: Chain-of-Thought Variants

- **Zero-shot CoT:** Trigger internal monologue with phrases like *"Let's think step by step."*
- **Few-shot CoT:** Provide examples that include the reasoning process, not just the final answer.
- **Tree-of-Thoughts (ToT):** Ask the model to explore multiple reasoning paths, self-critique them, and then choose the best one.
- **When to use:** Essential for logic puzzles, math problems, and multi-step planning where the process is as important as the result.

Reasoning Control Variants

- Plan-then-solve: short plan $\rightarrow$ final answer only.
- Critique-then-revise: rubric with 2–3 criteria; output revised version.
- Self-consistency: generate 3 variants, pick consensus elements.
- Debate-style: two brief personas; moderator summary only.

Retrieval-Augmented Prompting

- Ground answers with provided excerpts, tables, or references.
- Instruct the model to cite where each claim came from (line or paragraph IDs).
- Include a fallback: *If the answer is not in the context, say: not found in provided context*

RAG: Do's and Don'ts

- **Do:** Chunk context, label with IDs, and cap tokens.
- **Do:** Ask for citation markers per claim (e.g., [p3, §2]).
- **Don't:** Mix system instructions inside context blocks.
- **Don't:** Assume model will ignore irrelevant excerpts—state it.

Advanced RAG Techniques

- **Query Transformation:** Before searching, ask the LLM to rephrase the user's query into a better search term or a hypothetical document (HyDE).
- **Re-ranking:** Retrieve a larger set of documents (e.g., 10-20) and use a second, lightweight LLM call to rank them for relevance before feeding the top K to the final prompt.
- **Handling Contradictions:** When context sources conflict, instruct the model to highlight the discrepancy and report on both perspectives.

Iterative Refinement (Critique → Improve)

- Phase 1 (Draft): Generate a short draft that meets constraints.
- Phase 2 (Critique): Evaluate against rubric (accuracy, clarity, format, tone).
- Phase 3 (Revise): Produce the improved version only.
- Stops mode collapse and helps you converge on quality.

Common Anti-Patterns (Prompt Smells)

- Overloaded ask: too many goals in one turn → split steps.
- Hidden constraints: unstated data or evaluation rules.
- Vague audience: no register guidance → define reader profile.
- Conflicting directives: "be brief" and "include all details".
- Unbounded output: missing limits → set caps and priority.

Safety and Guardrails

- Add scope limits and disclaimers for sensitive domains.
- Disallow private data; require anonymization if provided.
- Refusal policy: specify when to decline and offer safe next steps.
- Ask for uncertainty notes when facts materially matter.

Anti-patterns and Safety

- Vague goals: *"Write something about X"* → Define purpose, length, audience.
- Hidden constraints: Share all required context; avoid trick questions.
- Overloaded tasks: Split multi-step tasks into sequential prompts.
- Unsafe asks: Add guardrails (no private data, no medical/financial advice, etc.).
- Over-trusting: Require citations or checks when accuracy matters.

Engineering Example

- **Task:** Summarize error logs and suggest next diagnostic steps for a web API.
- **Better prompt:** Role + structure + constraints + fallback.
- **Output:** Bulleted issues, probable causes, next 3 checks, and a one-line risk note.

Policy Example

- **Task:** Draft a 200-word executive brief on an AI bill using provided excerpts.
- **Prompt:** RAG citations per claim, JSON metadata: "risk": oneOf[low,med,high].
- **Output:** Brief paragraph + 3 action items with citation tags.

Education Example

- **Task:** Create a 15-minute lesson outline on photosynthesis for grade 8.
- **Prompt:** Audience and tone, 3 learning objectives, 2 activities, 5-question exit ticket.
- **Output:** Sections with timing; avoid jargon beyond grade level; include misconceptions to watch.

Medical Example

- **Task:** Draft a patient-friendly explanation of an HbA1c lab result.
- **Prompt:** Persona (diabetes educator), reading level (B1), structured bullets, safety disclaimer.
- **Output:** Definition, why it matters, what the number means, lifestyle pointers, when to contact a clinician.

Pattern: Prompt Chaining

Concept

Break a complex task into a pipeline of smaller, specialized prompts. The output of one prompt becomes the input for the next.

- **Prompt 1 (Extract):** Pull raw, unstructured information from a source document.
- **Prompt 2 (Structure):** Convert the raw text into a clean, predictable JSON object.
- **Prompt 3 (Summarize):** Use the structured JSON to generate a human-readable summary.
- **Benefits:** Improves reliability, makes debugging easier, and allows for specialized roles at each step.

Key Takeaways

- Clear intent, audience, and constraints make the biggest difference.
- Few-shot, role, structure, and reflection are versatile building blocks.
- Iterate: draft, critique, and revise to reach dependable quality.
- Add guardrails and citations when accuracy and safety matter.

Thank You!

Dr. Mehmet Ali Akyol

mehmetali.akyol@gmail.com

Questions?